

CSE327

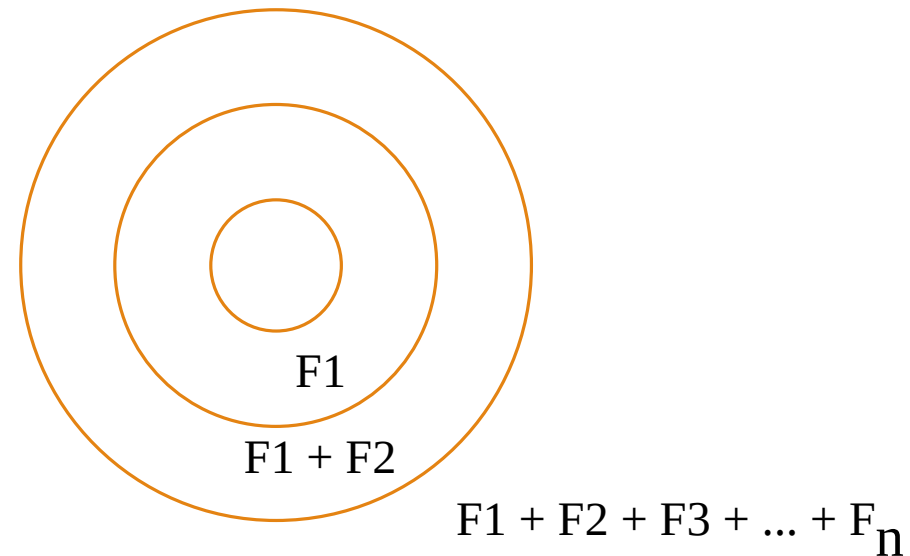
Software Engineering

AKM IQTIDAR NEWAZ [IQN]

LECTURE - 11

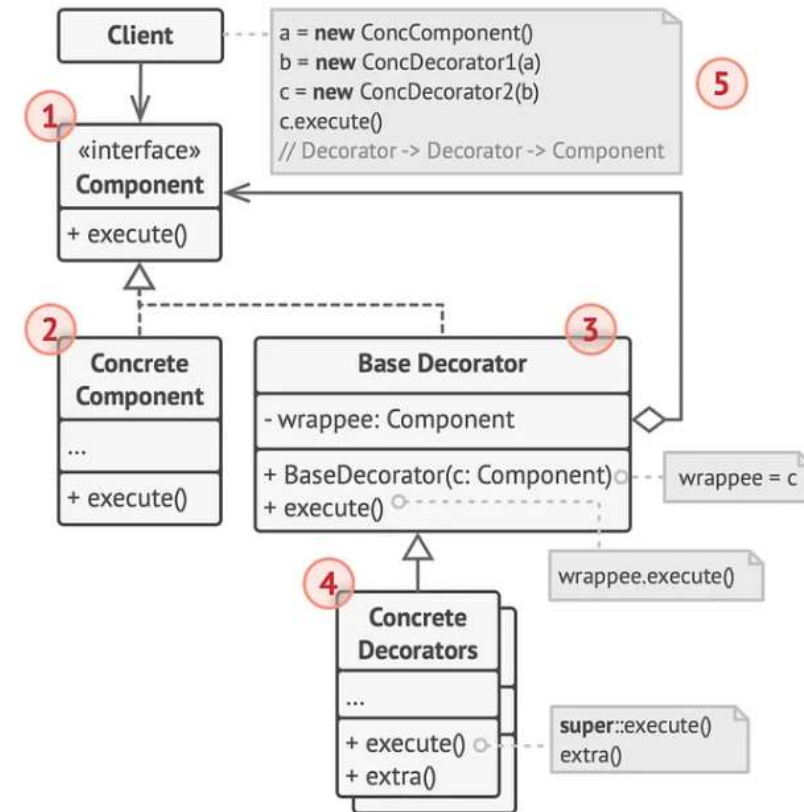
Decorator Design Pattern

- ❑ Decorator is a **structural design pattern** that lets you attach new behaviors to objects by placing these objects inside special wrapper objects that contains the behaviors.
- ❑ Suppose, we have a base object where that base object has some features/properties, and if we want to add more features the we need to use decorator for wrapping the F1 and it is also an object and we can still use a wrapper of decorator for adding new features.



UML Diagram of Decorator Pattern

- **Component** Interface:
 - Defines the base functionality.
 - Represented as an interface in UML.
- **ConcreteComponent** Class:
 - Implement the **Component** interface.
 - Provides the core functionality.
- **Decorator** Abstract Class:
 - Implement the Component interface.
 - Contains a reference to a **Component** object.
 - Adds functionality by delegating calls to the wrapped component
- **ConcreteDecorator** Class:
 - Extend the **Decorator** class.
 - Add specific behavior while maintaining the base functionality.



Basic Example - 1

- Consider a ***Window*** in a user interface as the Component.
- We want to add functionalities like scrollbars or a border without redesigning the window itself.
- In this example, ***SimpleWindow*** is a Concrete Component that can be decorated with additional features.
- ***WindowDecorator*** is a decorator that holds a reference to a Window object and delegates the draw operation to it.
- ***ScrollbarWindow*** is a Concrete Decorator that adds a scrollbar to the window.

Basic Example – 1 (Code)

```
#include <iostream>
using namespace std;
```

main.cpp

```
class Window // Component
{
public:
    virtual void draw() = 0;
    virtual ~Window() = default;
};

class SimpleWindow : public Window // Concrete Component
{
public:
    void draw() override
    {
        cout << "Drawing SimpleWindow()" << endl;
    }
};
```

```
class WindowDecorator : public Window // Decorator
{
public:
    WindowDecorator(Window *window) : window_(window) {}
```

```
    void draw() override
    {
        window_>draw(); // Delegate to the component
    }
```

main.cpp

```
protected:
    Window *window_;
};
```

```
class ScrollbarWindow : public WindowDecorator // Concrete Decorator
{
public:
    ScrollbarWindow(Window *window) : WindowDecorator(window) {}
```

```
    void draw() override
    {
        WindowDecorator::draw(); // Draw the window
        drawScrollbar();          // Add scrollbar
    }
```

```
private:
    void drawScrollbar()
    {
        cout << "Drawing ScrollbarWindow()" << endl;
    }
};
```

Basic Example – 1 (Code)

```
int main()
{
    cout << "Concrete Component:\n";
    Window *simpleWindow = new SimpleWindow();
    simpleWindow->draw();
    cout << "\nDecorator or Wrapper object:\n";
    Window *simpleWindow2 = new SimpleWindow();
    Window *scrollbar = new ScrollbarWindow(simpleWindow2);
    scrollbar->draw();
}
```

Output:

Concrete Component:
Drawing SimpleWindow()

Decorator or Wrapper object:
Drawing SimpleWindow()
Drawing ScrollbarWindow()

Basic Example – 2

- Let's consider a text editor application where users can write and format their text.
- Initially, the editor offers basic text operations.
- However, as an intermediate developer, we want to add functionalities like spell check, grammar check, and auto-correct without altering the core text processing component.
- In this example, ***BasicEditor*** provides fundamental text editing functionalities.
- Decorators like ***SpellCheckerDecorator*** add spell-checking capabilities on top of the basic text-writing functionality without changing the ***BasicEditor*** class.

Basic Example – 2 (Code)

main.cpp

```
#include <iostream>
#include <string>
using namespace std;

class TextEditor // Component
{
public:
    virtual void write(const string &words) = 0;
    virtual ~TextEditor() = default;
};

class BasicEditor : public TextEditor // Concrete Component
{
public:
    void write(const string &words) override
    {
        cout << "Performing Basic Editor Functionality!" << endl;
    }
};
```

main.cpp

```
class EditorDecorator : public TextEditor // Decorator
{
public:
    EditorDecorator(TextEditor *editor) : editor_(editor) {}

    void write(const string &words) override
    {
        editor_->write(words); // Delegate to the component
    }

protected:
    TextEditor *editor_;
};

class SpellCheckerDecorator : public EditorDecorator // Concrete Decorator
{
public:
    SpellCheckerDecorator(TextEditor *editor) : EditorDecorator(editor) {}

    void write(const string &words) override
    {
        EditorDecorator::write(words); // Write text
        spellCheck(words);             // Add spell checking
    }

private:
    void spellCheck(const string &words)
    {
        cout << "Performing the spell checking here!!" << endl;
    }
};
```

Basic Example – 2 (Code)

main.cpp

```
int main()
{
    TextEditor *editor = new BasicEditor();
    editor->write(" CSE-327 ");
    cout << "\nAfter performing Decorator wrapping:\n";
    TextEditor *editor2 = new BasicEditor();
    SpellCheckerDecorator *spellChecker = new
SpellCheckerDecorator(editor2);
    spellChecker->write(" CSE-327 ");
}
```

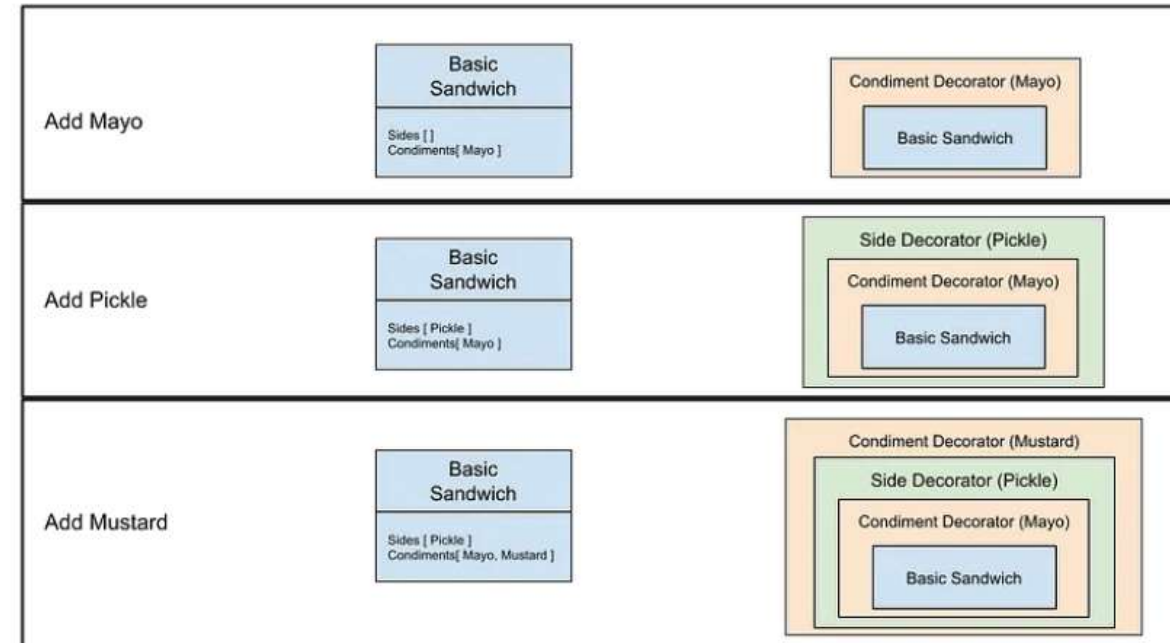
Output:

Performing Basic Editor Functionality!

After performing Decorator wrapping:
Performing Basic Editor Functionality!
Performing the spell checking here!!

Example – 3 (Order Sandwich)

- Suppose we're building a sandwich shop where the user has the ability to choose the base of the sandwich, then add sides and condiments onto it.
- The base of the sandwich has a basic version, which uses regular bread and meat, where as the deluxe version uses organic bread and meat, along with cheese and lettuce in it.
- After the customer choose the base, and selects the additional sides and condiments, the sandwich shop will return the total cost and the various ingredients that went into sandwich.



Example – 3 (Code)

```
#pragma once
#include <string>
#include <iostream>
using namespace std;

class SandwichOrder // Component
{
public:
    virtual int GetCost() const = 0;
    virtual string GetIngredient() const = 0;
};
```

sandwichOrder.h

```
#pragma once
#include "sandwichOrder.h"

class BasicSandwich : public SandwichOrder // Concrete
Component
{
public:
    int cost = 5;
    int GetCost() const override
    {
        return cost;
    }
    string GetIngredient() const override
    {
        return "Basic Bread and Meat";
    }
};
```

basicSandwich.h

Example – 3 (Code)

```
#pragma once
#include "sandwichOrder.h"

class DeluxeSandwich : public SandwichOrder // Concrete Component
{
public:
    int cost = 8;
    int GetCost() const override
    {
        return cost;
    }
    string GetIngredient() const override
    {
        return "Organic Bread, Organic Meat, Cheese, veggie";
    }
};
```

deluxeSandwich.h

```
#pragma once
#include "sandwichOrder.h" decorator.h

class Decorator : public SandwichOrder // Decorator
{
public:
    SandwichOrder *order_;

    Decorator(SandwichOrder *order) : order_(order) {}
    int GetCost() const override
    {
        return order_->GetCost();
    }

    string GetIngredient() const override
    {
        return order_->GetIngredient();
    }
};
```

Example – 3 (Code)

```
#pragma once
#include "decorator.h"

class CondimentDecorator : public Decorator
{
public:
    string condiment_;
    int cost_ = 1;

    CondimentDecorator(string condiment, SandwichOrder *order) : Decorator(order) {}
    condiment_(condiment), Decorator(order) {}

    int GetCost() const override
    {
        return cost_ + Decorator::GetCost();
    }

    string GetIngredient() const override
    {
        return condiment_ + " " + order_->GetIngredient();
    }
};
```

condimentDecorator.h

```
#pragma once
#include "decorator.h"

class SideDecorator : public Decorator
{
public:
    string side_;
    int cost = 2;
    SideDecorator(string side, SandwichOrder *order) : side_(side),
    Decorator(order) {}

    int GetCost() const override
    {
        return cost + Decorator::GetCost();
    }

    string GetIngredient() const override
    {
        return side_ + " " + order_->GetIngredient();
    }
};
```

sideDecorator.h

Example – 3 (Code)

main.cpp

```
#include "condimentDecorator.h"
#include "sideDecorator.h"
#include "deluxeSandwich.h"
#include "basicSandwich.h"
#include <iostream>
using namespace std;

void ServeOrder(SandwichOrder *order)
{
    std::cout << "Total Cost: " << order->GetCost() << std::endl;
    std::cout << "Ingredients: " << order->GetIngredient() << std::endl;
}

int main()
{
    SandwichOrder *sandwich1 = new DeluxeSandwich;
    SandwichOrder *decorated1 = new CondimentDecorator("mayo", sandwich1);
    SandwichOrder *decorated2 = new SideDecorator("pickle", decorated1);
    SandwichOrder *decorated3 = new CondimentDecorator("mustard",
decorated2);
    SandwichOrder *final_order = new SideDecorator("onion", decorated3);
    ServeOrder(final_order);
    return 0;
}
```

Output:

Total Cost: 14

Ingredients: onion mustard pickle mayo Organic Bread, Organic Meat, Cheese, veggie